

Session 02 – Convolutional Neural Networks

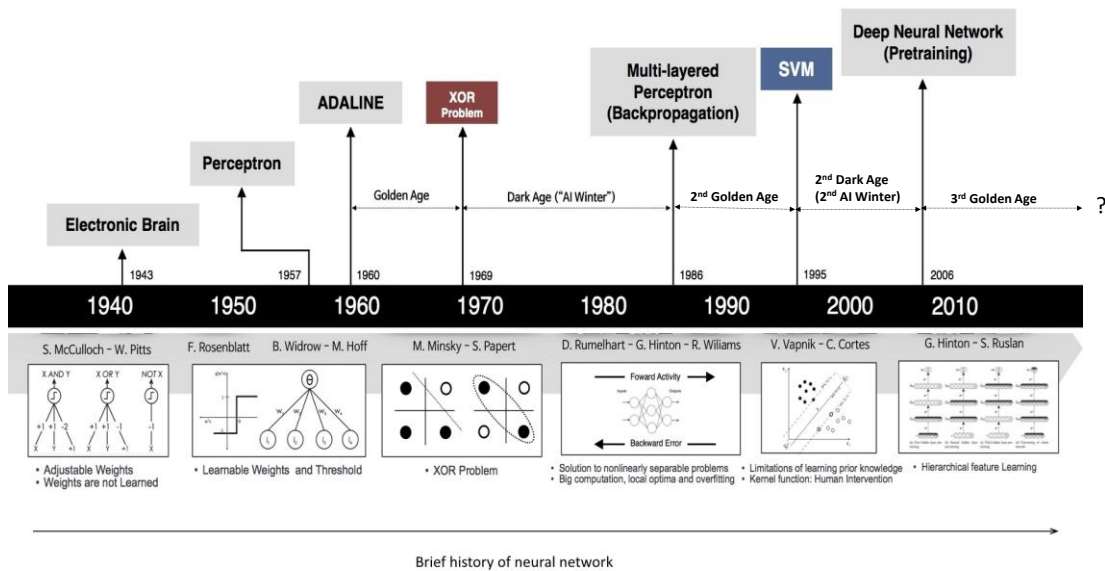
Dr Ivan Olier

I.Olier@ljmu.ac.uk

2026 Summer School in AI

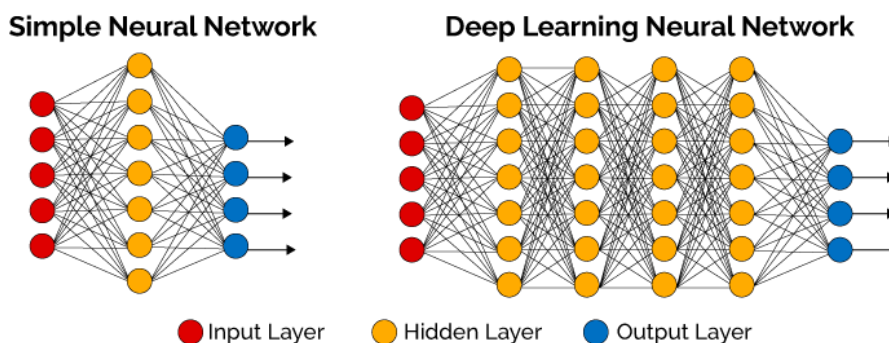
1

Brief history of neural network



2

From Neural Networks to Deep Learning

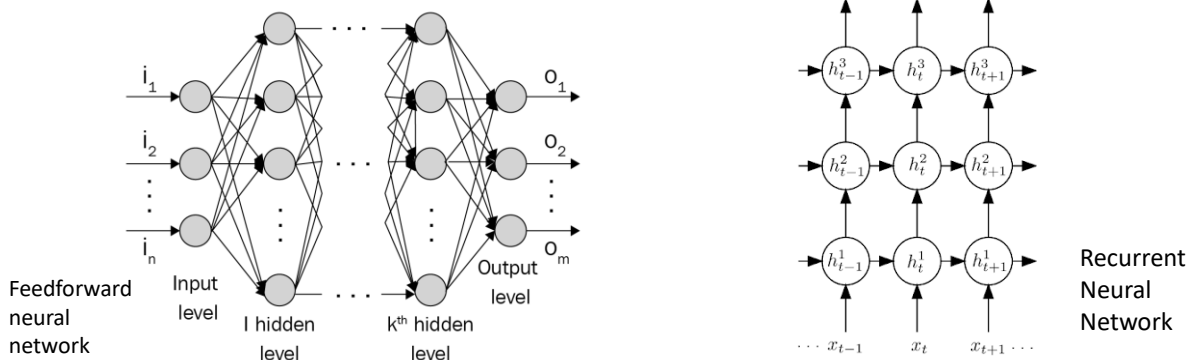


- Deep learning is actually a rebranding of artificial neural networks, but the term is usually reserved to architectures with several (many) layers.

3

Deep learning architectures

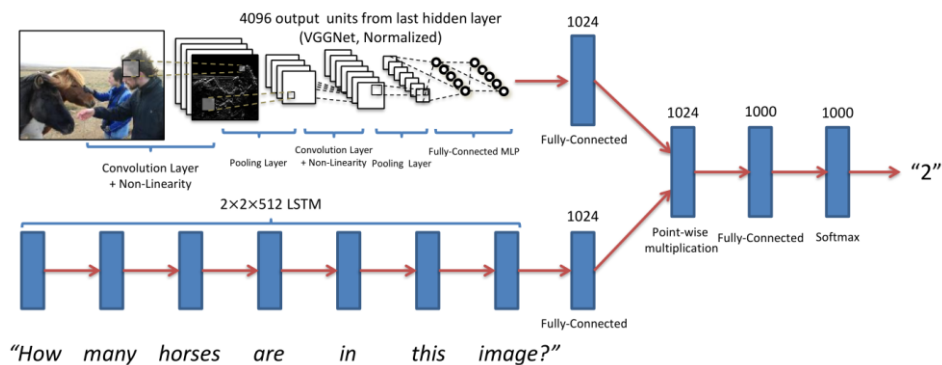
- We have two main families of architectures:
 - Feedforward networks – learn to map inputs to outputs (e.g. dense or fully connected neural networks (DNNs) and convolutional neural networks (CNNs))
 - Recurrent networks (ReNNs) – outputs are determined from the inputs and current state of the hidden units (memory). A popular ReNNs are the Long-Short Term Memories (LSTMs)
- They usually learn by error backpropagation.



4

Deep learning architectures

- A deep-learning architecture is a multilayer stack of simple modules, all (or most) of which are subject to learning, and many of which compute non-linear input–output mappings.
- It is becoming popular deep learning architectures with 15 to 20 or more hidden layers.

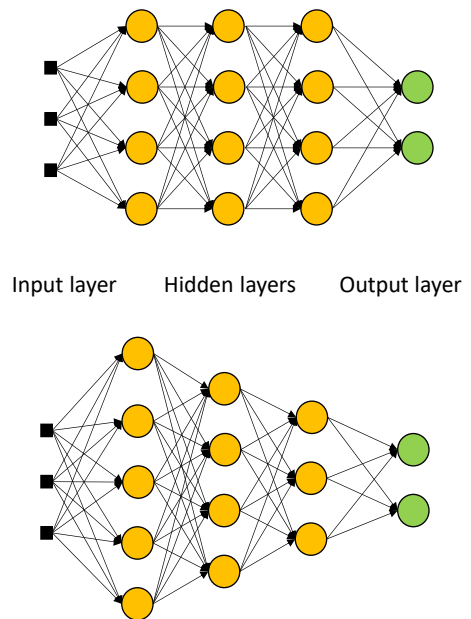


Dense feedforward networks

- Or fully connected networks – full connectivity between adjacent layers.
- They are general purpose networks – they do not specialise to any particular data type.
- The Rectified Linear Units (ReLUs, $f(v) = \max(v, 0)$) are the most popular activation functions for the hidden neurons. They typically learn faster than sigmoids in networks with many layers.
- Although there is the risk of getting trapped in poor local minima, in practice they are rarely a problem with large networks.

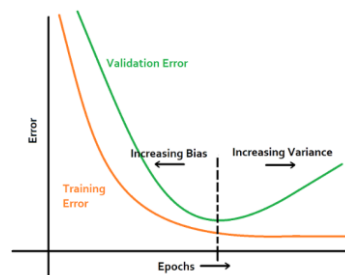
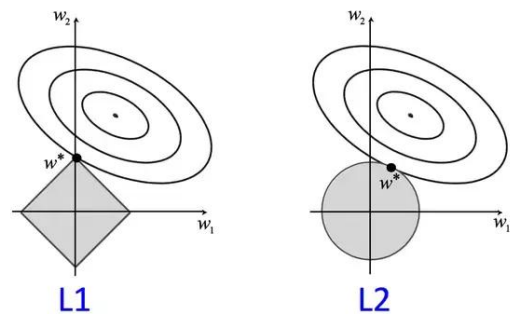
Hidden layers

- Number of hidden layers is highly dependent on the problem.
- Generally, 1–5 hidden layers will serve you well for most problems.
- In general, using the same number of neurons for all hidden layers will suffice. For some datasets, having a large first layer and following it up with smaller layers will lead to better performance as the first layer can learn a lot of lower-level features that can feed into a few higher order features in the subsequent layers.
- Usually, you will get more of a performance boost from adding more layers than adding more neurons in each layer.



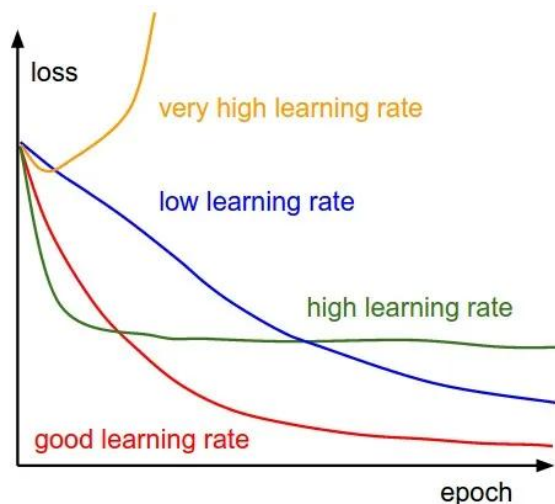
Learning

- Try “**overfit then regularise**” approach
- Regularisation:
 - Get more data
 - *Data augmentation*
 - *Pretraining*
 - Dimensionality reduction / Feature selection
 - Smaller model size
 - Smaller batch size
 - Drop out neurons
 - L1 and L2 regularisations
 - Early stopping



Learning

- **Batch size:**
 - Large batch sizes serve well for scaling and parallelisability.
 - Small batch sizes increase performance generalisation.
 - Recommendation: start with mini-batch sizes between 2 and 32, then increase.
- **Scaling:** Always scale your features.
- **Learning rate:** start very low (e.g. 10^{-6}) then increase.



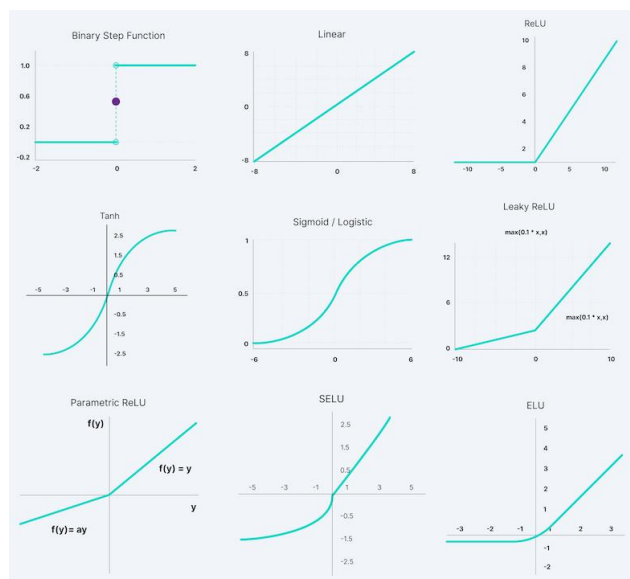
Activation functions

Hidden Layers:

- In general, the performance from using different activation functions improves in this order :
logistic $\rightarrow$ tanh $\rightarrow$ ReLU $\rightarrow$ Leaky ReLU $\rightarrow$ ELU $\rightarrow$ SELU.
- If you don't want to tweak your activation function, ReLU is a great place to start.

Output Layer

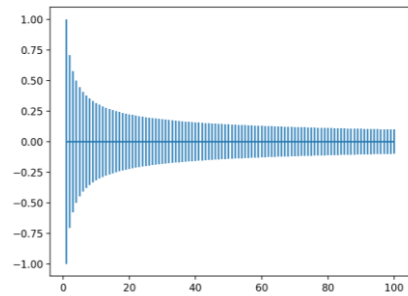
- **Regression:**
 - Linear (general case)
 - Tanh/Sigmoid (bounded into a certain range)
 - Softplus (positive values only)
- **Classification:**
 - Sigmoid (binary classification)
 - Softmax (multi-class classification)



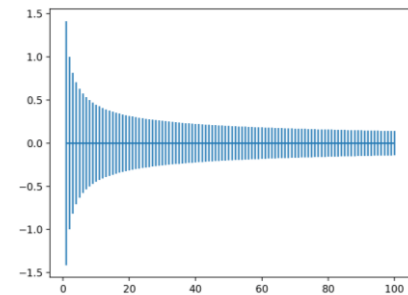
Weight initialisations

- Random initialisation
- Xavier/Glorot weight initialisation
 - $w = U \left[-\frac{1}{\sqrt{n}}, \frac{1}{\sqrt{n}} \right]$
 - Used for **sigmoid/tanh/softmax**
- He weight initialisation
 - $w = N \left[0, \sqrt{\frac{2}{n}} \right]$
 - Used for **ReLU/Leaky ReLU**
- n is the number of inputs to the node.

Xavier



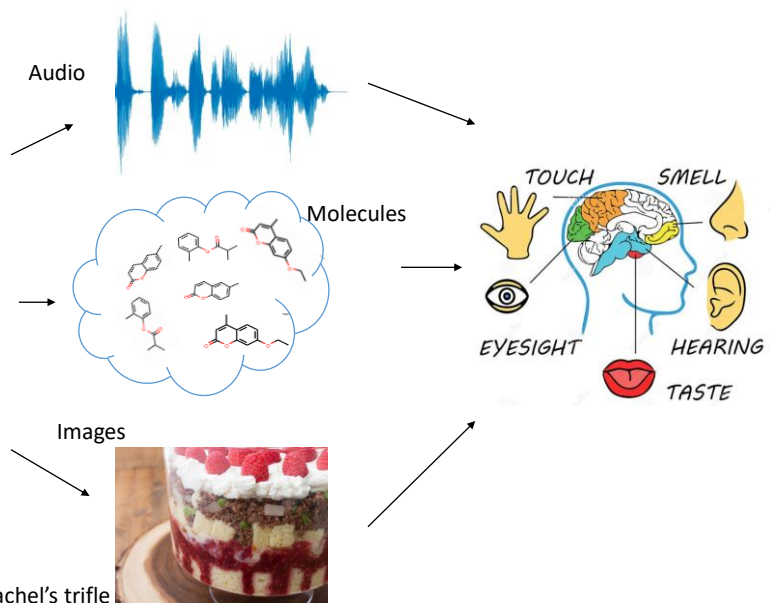
He



11

Natural signals

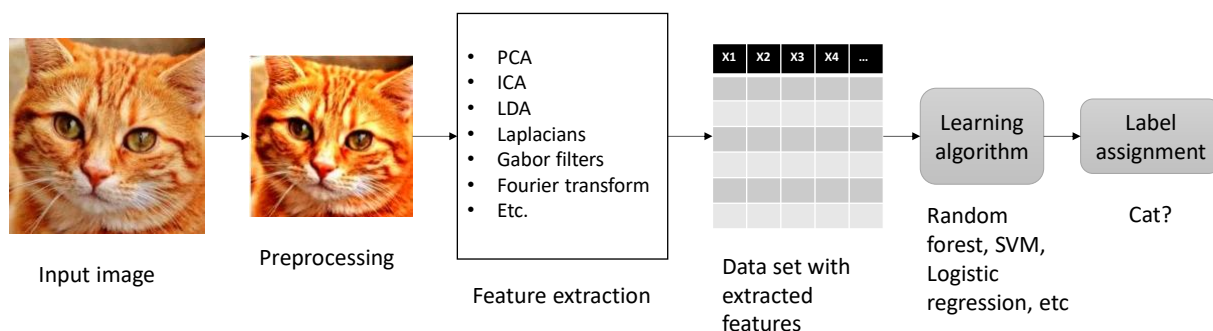
- We receive information in formats different from just tabular data.
- We learn how to extract features from those data



12

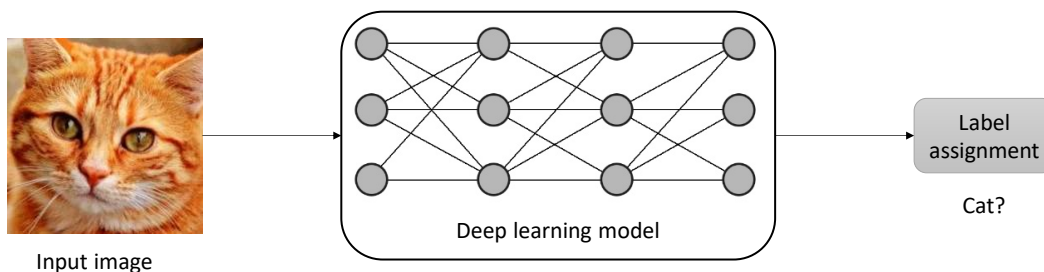
Conventional machine learning

- Conventional machine-learning techniques were limited in their ability to process natural data in their raw form.
- Constructing a pattern-recognition or machine-learning system required careful engineering and considerable domain expertise to design a feature extractor that transformed the raw data (such as the pixel values of an image) into a suitable internal representation or feature vector from which the learning subsystem, often a classifier, could detect or classify patterns in the input.



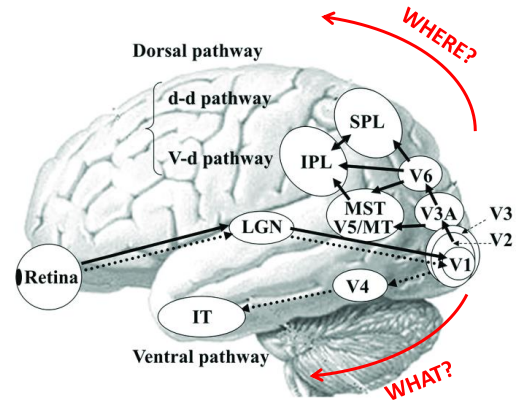
Representation learning methods

- Representation learning is a set of methods that allows a machine to be fed with raw data and to automatically discover the representations needed for detection or classification.
- Deep-learning methods can be representation-learning methods with multiple levels of representation, obtained by composing simple but non-linear modules that each transform the representation at one level (starting with the raw input) into a representation at a higher, slightly more abstract level. With the composition of enough such transformations, very complex functions can be learned.
- The key aspect of deep learning is that these layers of features are not designed by human engineers: they are learned from data using a general-purpose learning procedure.



Visual cortex model

- Visual cortex is a region in the occipital lobe of the brain which processes visual information.
- Visual information coming from the **eyes** goes through the **thalamus** and then reaches the **visual** cortex.
- Visual cortex response is **localised**, with clear divisions and functionalities.
- It is divided into several visual areas:
 - V1 (primary visual cortex) – creates **saliency maps** to highlight what is important to guide the shift of attention.
 - V2 (secondary visual cortex), V3, and V4 – are tuned for **orientation, spatial frequency, and colour**. The difference is in the tuning for object feature complexity.
 - V5 – perception of **motion**
 - V6 – perception of **self-motion**



Convolutional Neural Networks (CNNs)

- Fully connected networks tend to learn **slowly** some datasets such as images.
- One simple solution to this problem is to **restrict** the connections between the hidden units and the input units, allowing each hidden unit to connect to only a small subset of the input units.
- This idea of having **locally connected** networks also draws inspiration from how the early **visual system** is wired up in biology.
- Specifically, neurons in the visual cortex have **localised receptive fields** (i.e., they respond **only** to stimuli in a certain location).
- CNNs are feedforward NNs, so they learn using similar **error backpropagation** algorithms.
- Note that if we are modelling a classification or regression task, CNNs will at least need a dense layer for the output.



Convolution

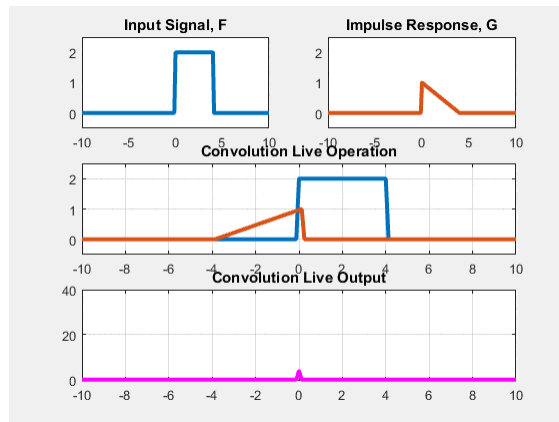
- Convolution is a mathematical operation on two functions that produces a third function expressing how the shape of one is modified by the other. It is defined as the integral of the product of the two functions after one is reversed and shifted.

Continuous-valued functions

$$y(t) = (x * h)(t) = \int_{-\infty}^{\infty} x(t)h(t - \tau)d\tau$$

Discrete-valued functions

$$y[n] = (x * h)[n] = \sum_{n'=-\infty}^{\infty} x[n]h[n - n']$$



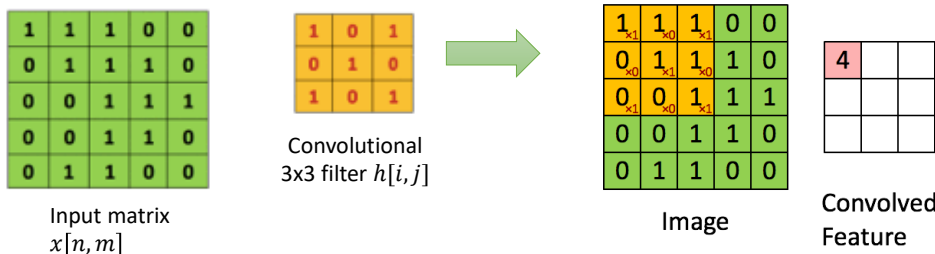
Convolutional filter

- A convolutional filter applies the convolution explicitly.
- Its operation is localised if $h[i, j]$ is zero for most of its range.

$$y[n, m] = \sum_{j=-\infty}^{\infty} \sum_{i=-\infty}^{\infty} x[n, m]h[n - i, m - j]$$

$$h[i, j] = \begin{cases} h[i, j] & i, j < I, J \\ 0 & \text{otherwise} \end{cases}$$

- It is a discrete filter, that operates on discrete signals, so it is very simple to implement.



- The convolution filter has the effect of detecting **motifs**, which are groups of values that are highly correlated.

Hand-crafted convolutional image filters

- A filter is a function $h(t)$, or $h[n]$, if continuous or discrete, designed to make a specific operation on another function $x(t)$ or $x[n]$, respectively.
- Image filters are filters $h[m, n]$ designed to operate on images:
- For instance:



- Low-pass filter – to smooth an image

1/9	1/9	1/9
1/9	1/9	1/9
1/9	1/9	1/9

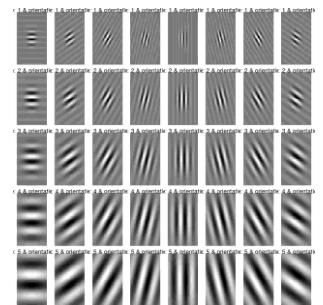
$h[m, n]$

- High-pass filter – to detect edges

-1	-1	-1
-1	9	-1
-1	-1	-1

$h[m, n]$

- Gabor filters – very popular filters to detect orientation. Gabor filters work similar to the primary visual cortex (V1)



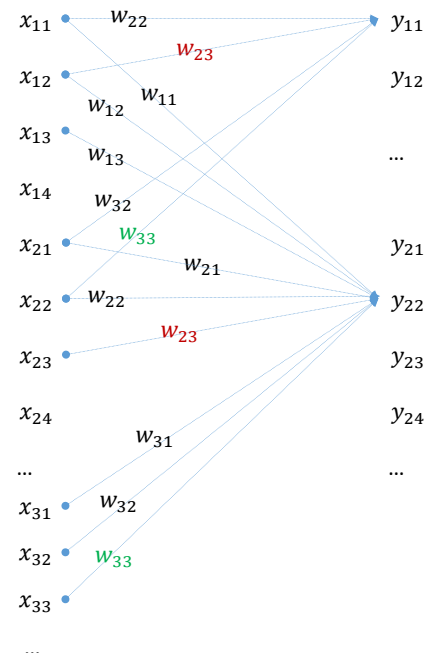
Convolutional filter

- A convolutional filter can be represented as a feedforward layer.

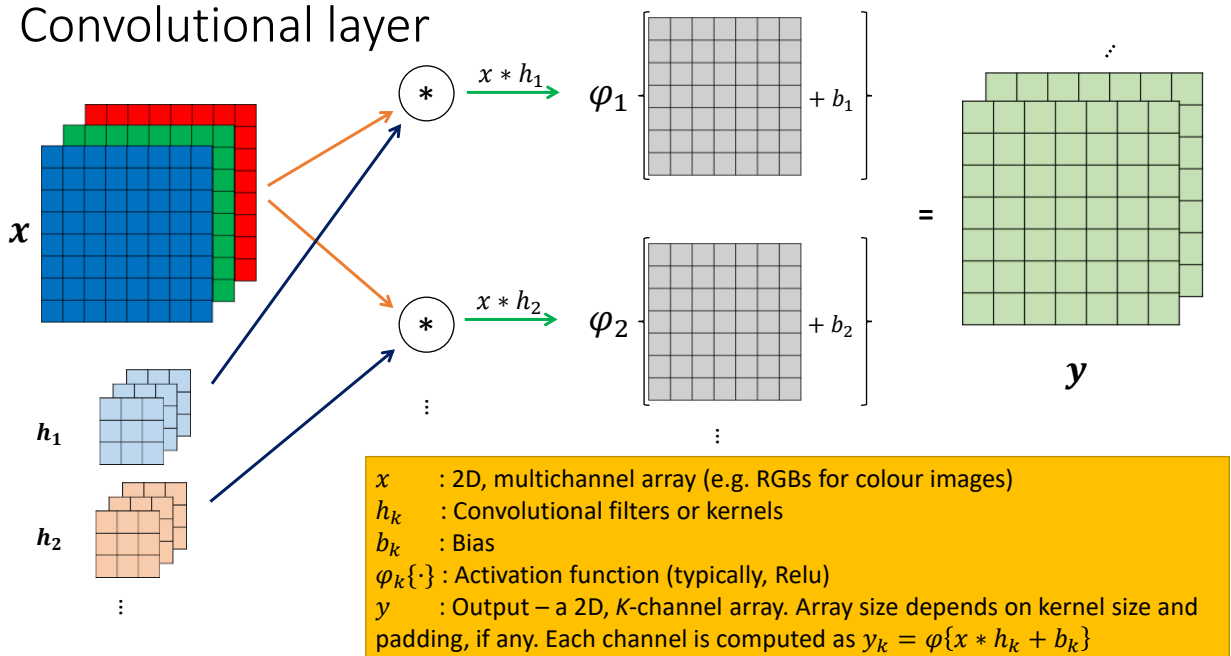
w_{11}	w_{12}	w_{13}
w_{21}	w_{22}	w_{23}
w_{31}	w_{32}	w_{33}

$$y_{nm}^{[t]} = x_{n-1,m-1}^{[t]}w_{11} + x_{n-1,m}^{[t]}w_{12} + x_{n-1,m+1}^{[t]}w_{13} + x_{n,m-1}^{[t]}w_{21} + x_{n,m}^{[t]}w_{22} + x_{n,m+1}^{[t]}w_{23} + x_{n+1,m-1}^{[t]}w_{31} + x_{n+1,m}^{[t]}w_{32} + x_{n+1,m+1}^{[t]}w_{33}$$

- With some particular properties:
 - Weight sharing within the layer
 - Neurons are locally connected
 - Filter coefficients become weights that are learnt from data.

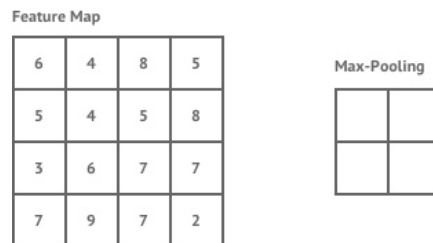


Convolutional layer



Max-pooling filter

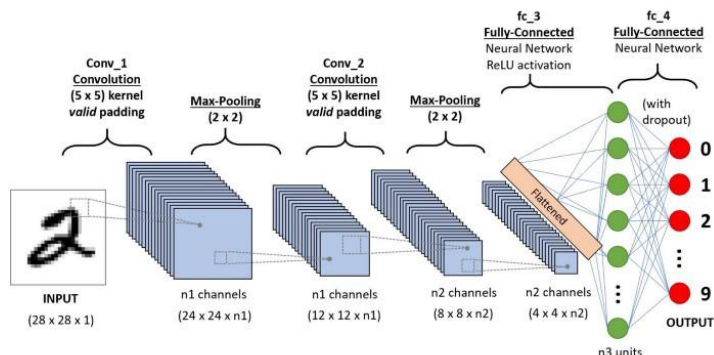
- A max-pooling filter computes the maximum of a local patch of values in a matrix (data, image, etc).
- It reduces the dimension of the original matrix too.



- Max-pooling after convolution – merges similar neighbouring motifs into one so it creates an invariance to small shifts and distortions.

Convolutional neural networks (CNNs)

- A typical architecture for image classification using CNNs + DNNs for image classification:



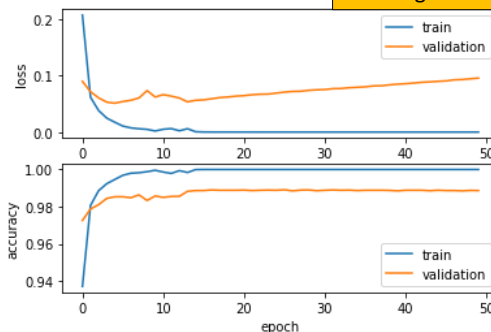
- It uses convolutional and max-pooling layers to extract features from images, followed by dense (fully-connected) layers to map them to classes.
- A flattening operation is needed to convert the output of the CNN part into the input vector of the first dense layer. It basically concatenates the array rows (or columns).

23

Implementing MNIST problem with CNNs

Model: "sequential"		
Architecture		
Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
flatten (Flatten)	(None, 5408)	0
dense (Dense)	(None, 512)	2769408
dense_1 (Dense)	(None, 10)	5130
Total params: 2,774,858		
Trainable params: 2,774,858		
Non-trainable params: 0		

Learning curves



Training

```
Epoch 48/50
375/375 - 2s - loss: 1.6430e-07 - accuracy: 1.0000 - val_loss: 0.0931 - val_accuracy: 0.9885 - 2s/epoch - 5ms/step
Epoch 49/50
375/375 - 1s - loss: 1.3695e-07 - accuracy: 1.0000 - val_loss: 0.0944 - val_accuracy: 0.9887 - 1s/epoch - 4ms/step
Epoch 50/50
375/375 - 1s - loss: 1.1390e-07 - accuracy: 1.0000 - val_loss: 0.0956 - val_accuracy: 0.9887 - 1s/epoch - 4ms/step
```

Model evaluation

```
Test loss model 1: 0.08048252761363983
Test accuracy model 1: 0.9886999726295471
Train loss model 1: 0.019123613834381104
Train accuracy model 1: 0.9977333545684814
```

24

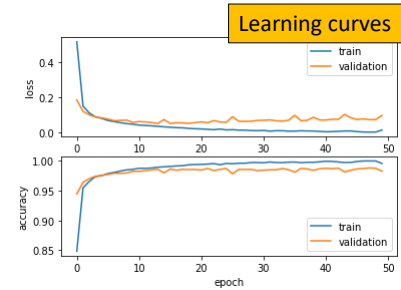
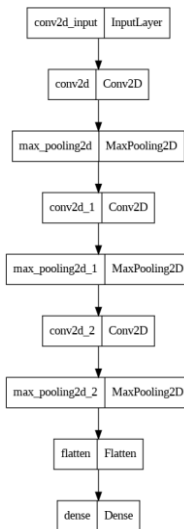
Version 2

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 32)	9248
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 32)	0
conv2d_2 (Conv2D)	(None, 3, 3, 32)	9248
max_pooling2d_2 (MaxPooling2D)	(None, 1, 1, 32)	0
flatten (Flatten)	(None, 32)	0
dense (Dense)	(None, 10)	330

=====
 Total params: 19,146
 Trainable params: 19,146
 Non-trainable params: 0
 =====

Architecture



Implementing Early Stopping with TF

```

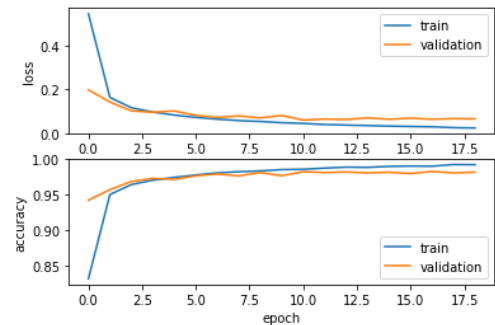
custom_early_stopping = EarlyStopping(
    monitor='val_loss',
    patience=8,
    min_delta=0.001
)

```

```

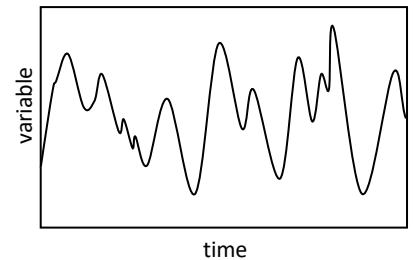
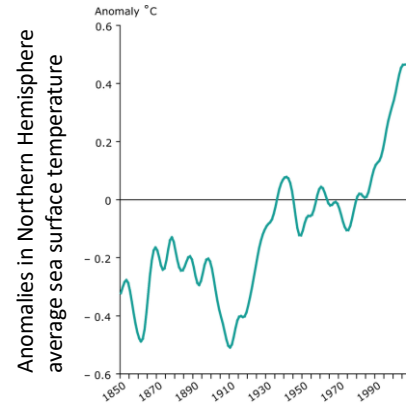
history = model1.fit(X_train, y_train,
    batch_size=128, epochs=100,
    verbose=2,
    validation_split=0.2,
    callbacks=[custom_early_stopping]
)

```



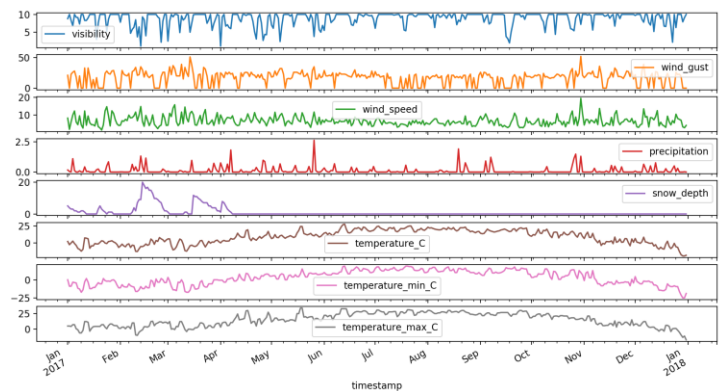
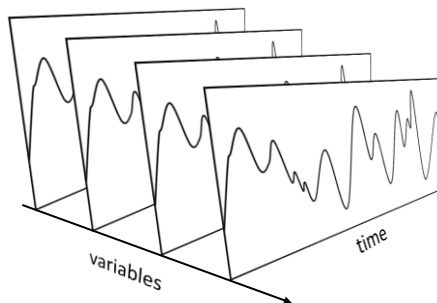
Time series

- A time series is a series of points indexed in **time order**.
- Time series are **produced** by dynamical systems.
- They are used in many application domains such as:
 - econometrics & finance;
 - weather, climate & earth science;
 - physiological signals & medicine;
 - control engineering;
 - astrophysics;
 - communications; and
 - in general, any **STEM** domain which involves temporal measurements
- We have an univariate time series (or simple, a time series) when the points are univariate or scalar.



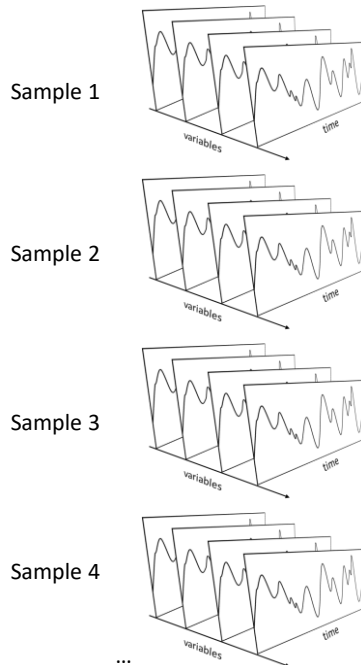
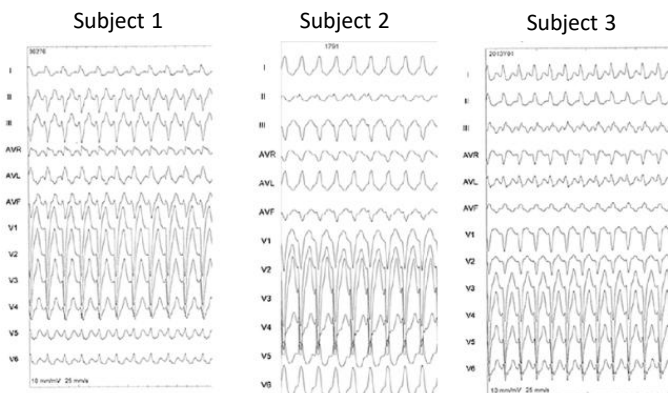
Multivariate time series

- Points in multivariate time series are **vectors**.
- This means that a point has **more than one** variable.
- It is usually assumed that all the variables are associated in some way. For instance, sensors collecting signals from a machine.



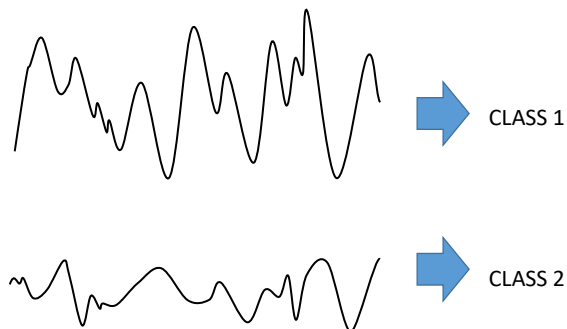
Multiple (multivariate) time series

- We have multiple univariate or multivariate time series when we have **more than one sample** available.
- e.g. Electrocardiograms of several subjects

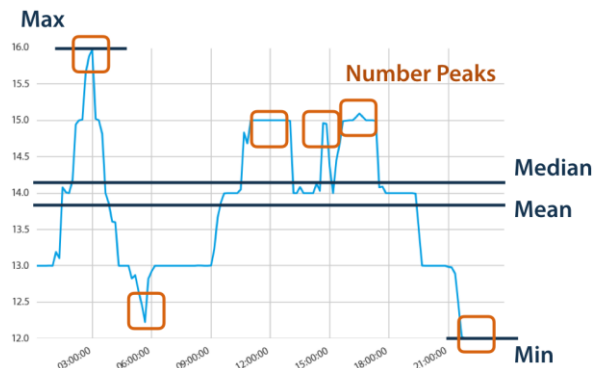


Time series classification

- Time series classification is simply the extension of the classification task to time series



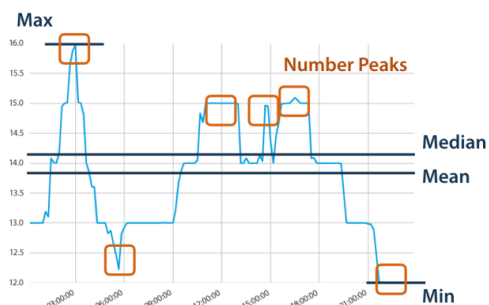
Extracting statistical features from time series



- There are many features that can be extracted from time series.
- e.g.: within a time window:
 - Maximum value
 - Minimum value
 - Number of peaks
 - Mean value
 - Median value
 - Standard deviation
 - Skew
 - Entropy
 - Area under the curve
- Some are specific to the problem:
 - In ECGs: R-R period, P-wave detection, etc

31

Time series classification using hand-crafted features



Max	Min	Med	...	Class
0.1	0.2	0.3	...	A
0.2	0.3	0.4	...	A
0.3	0.4	0.5	...	B
0.4	0.5	0.6	...	A
0.5	0.6	0.7	...	B
0.6	0.7	0.8	...	B
0.7	0.8	0.9	...	B

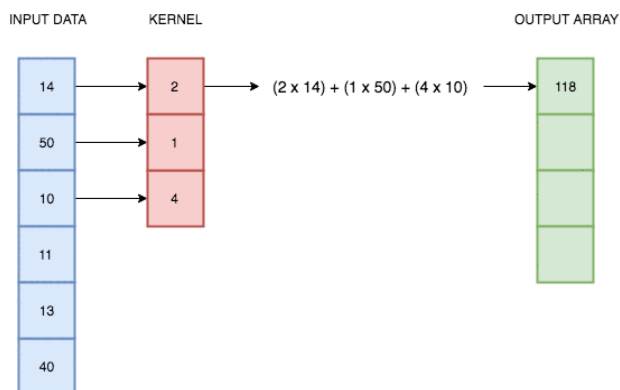
Traditional Machine Learning
pipeline (e.g. you can use
random forest, SVMs, etc)



32

Using CNNs to model time series

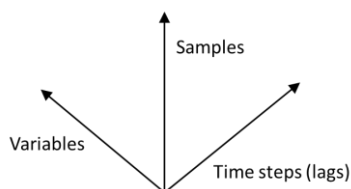
- Multivariate time series can also be modelled using CNNs



33

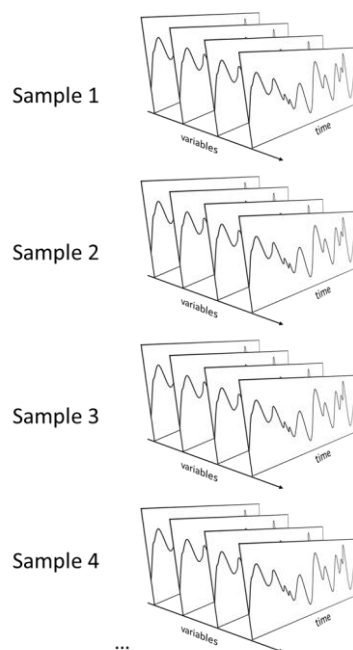
Using CNNs to model time series

- Multiple, multivariate times series can be modelled with CNNs as follows:



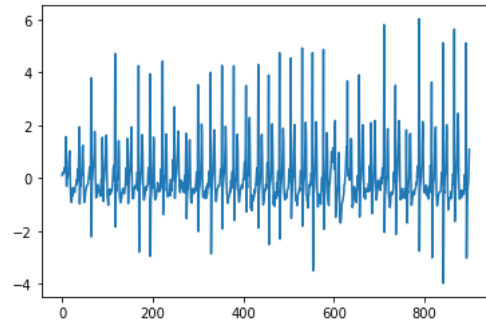
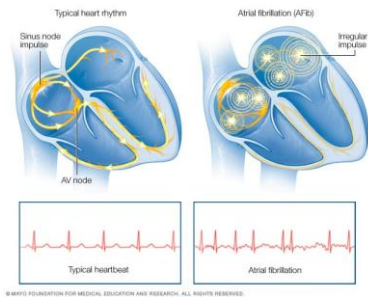
- Lags are controlled by the 1D convolutional filter length.
- Variables are the filter channels.
- Samples are the observations.

- In principle, CNNs should not be used for time series forecasting
 - ... 1D convolutional filters use past and future to predict present values.
 - However, there are variants that could be used for forecasting too.



34

Application example



- Atrial Fibrillation (AF) is the **commonest** arrhythmia worldwide and **increases** the risk of stroke and heart failure.
- AF is characterised by **irregular** atrial electric activity and ventricular response.
- AF is typically **detected** using ECGs.

Waveforms (ECGs):

- 3 classes:
 - Other rhythms
 - Normal sinus
 - Atrial fibrillation
- 6,072 waveforms
- 900 time points

35

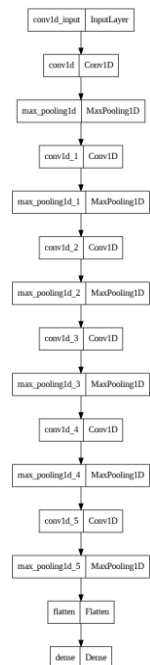
Implementation in TF

Architecture

```
ecg_md1 = Sequential()
ecg_md1.add(layers.Conv1D(filters=16, kernel_size=8,
padding='same',
activation='relu', input_shape=(900,1)))
ecg_md1.add(layers.MaxPool1D(pool_size=3))
ecg_md1.add(layers.Conv1D(filters=16, kernel_size=8,
padding='same', activation='relu'))
ecg_md1.add(layers.MaxPool1D(pool_size=3))
ecg_md1.add(layers.Conv1D(filters=16, kernel_size=8,
padding='same', activation='relu'))
ecg_md1.add(layers.MaxPool1D(pool_size=3))
ecg_md1.add(layers.Conv1D(filters=16, kernel_size=8,
padding='same', activation='relu'))
ecg_md1.add(layers.MaxPool1D(pool_size=3))
ecg_md1.add(layers.Conv1D(filters=16, kernel_size=8,
padding='same', activation='relu'))
ecg_md1.add(layers.MaxPool1D(pool_size=3))
ecg_md1.add(layers.Flatten())
ecg_md1.add(layers.Dense(3, activation='softmax'))
```

Layer (type)	Output Shape	Param #
conv1d (Conv1D)	(None, 900, 16)	144
max_pooling1d (MaxPooling1D)	(None, 300, 16)	0
conv1d_1 (Conv1D)	(None, 300, 16)	2064
max_pooling1d_1 (MaxPooling1D)	(None, 100, 16)	0
conv1d_2 (Conv1D)	(None, 100, 16)	2064
max_pooling1d_2 (MaxPooling1D)	(None, 33, 16)	0
conv1d_3 (Conv1D)	(None, 33, 16)	2064
max_pooling1d_3 (MaxPooling1D)	(None, 11, 16)	0
conv1d_4 (Conv1D)	(None, 11, 16)	2064
max_pooling1d_4 (MaxPooling1D)	(None, 3, 16)	0
conv1d_5 (Conv1D)	(None, 3, 16)	2064
max_pooling1d_5 (MaxPooling1D)	(None, 1, 16)	0
flatten (Flatten)	(None, 16)	0
dense (Dense)	(None, 3)	51

Total params: 10,515
 Trainable params: 10,515
 Non-trainable params: 0



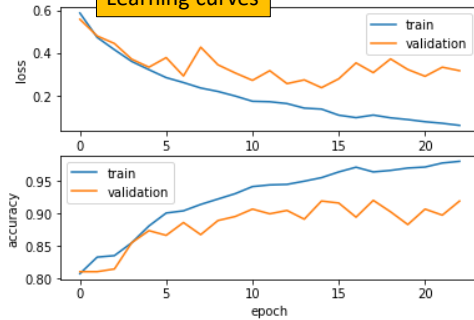
36

Model outputs

Training

```
Epoch 21/100
122/122 [=====] - 1s 8ms/step - loss: 0.0783 - accuracy: 0.9707 - val_loss: 0.2908 - val_accuracy: 0.9064
Epoch 22/100
122/122 [=====] - 1s 8ms/step - loss: 0.0710 - accuracy: 0.9768 - val_loss: 0.3333 - val_accuracy: 0.8971
Epoch 23/100
122/122 [=====] - 1s 8ms/step - loss: 0.0612 - accuracy: 0.9794 - val_loss: 0.3169 - val_accuracy: 0.9187
```

Learning curves



```
ecg_md1.evaluate(x=X_test, y=y_test)
```

```
38/38 [=====] - 0s 7ms/step - loss: 0.1632 - accuracy: 0.9424
[0.1632407009601593, 0.9423868060112]
```

Model evaluation

```
[[ 112  21  4]
 [  19 1002  5]
 [   8  13 31]]
```

Summary

- We learnt about Deep Learning and how it relates to Machine Learning and AI.
- We studied CNNs, which are Feedforward Neural Networks.
- Designed to handle time series (Conv1D), images (Conv2D), videos (Conv3D)

